DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING • DEVOPS • GROUP 33

End-to-End DevOps Lifecycle & Cloud-Native Deployment

PBL Project: RepoWise (AI Codebase Intelligence Platform)

A comprehensive demonstration of Automated CI/CD Pipelines, Infrastructure as Code, Containerization, Kubernetes Orchestration with Zero-Downtime Rollouts, and Prometheus Telemetry.

CONTINUOUS ASSESSMENT II (CA-II) • GROUP 33 TEAM MEMBERS

Kshitiz Shah	Nitesh Ghimire	Nigel Francy	Omkar Kadam
PRN: 23070122121	PRN: 23070122150	PRN: 23070122148	PRN: 23070122158

System Architecture Overview

Multi-Tier Microservices Decomposition & Target Scope

ARCHITECTURE

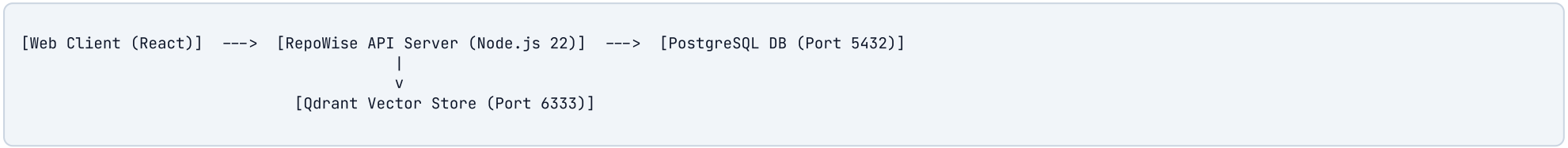
RepoWise is an AI-powered repository indexing and codebase intelligence system. The platform is designed using a decoupled microservices pattern to ensure independent scaling, modular maintenance, and fault isolation.

🏗️ Microservice Architecture Decomposition

- **Frontend Client Tier (`client/`):** React / Vite web interface served via high-performance Nginx.
- **Core API Server (`server/` — Target Service):** Node.js 22 + TypeScript + Express + Prisma ORM.
- **AI Orchestration Service (`ai-service/`):** Python FastAPI handling embeddings and vector search.
- **Storage Tier:** PostgreSQL (relational metadata) + Qdrant (vector index).

🎯 DevOps Scope & Target Service

- **Selected Service:** `@repowise/server` backend microservice.
- **Role in Platform:** Coordinates authentication, database transactions, vector queries, and API endpoints.
- **DevOps Pipeline Target:** Selected for automated testing, containerization, Ansible provisioning, and Kubernetes rolling updates.



CI/CD Pipeline Flow (GitHub Actions)

Automated Testing, Multi-Stage Containerization & Continuous Delivery

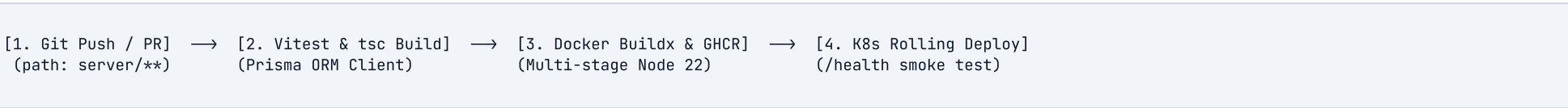
PIPELINE

⚙️ Pipeline Execution Stages

- **1. Smart Trigger & Filter:** Triggers on push/PR to `main` with path filtering (`server/**` only), preventing redundant runs.
- **2. Continuous Integration:** Runs on `ubuntu-latest`, setups Node.js 22, generates Prisma client, runs Vitest suite, and compiles TypeScript.
- **3. Container Packaging:** Builds multi-stage Docker image, authenticates with GHCR, tags with `:latest` and `:sha-xxx`.
- **4. Deployment & Verification:** Triggers Kubernetes rolling update and executes automated `/health` probe.

🔒 Engineering & Security Rationale

- **Zero Credential Exposure:** Authenticates via short-lived GitHub-native `${{ secrets.GITHUB_TOKEN }}`.
- **Quality Gates:** Fails the build immediately if any unit test or TypeScript compile check errors.
- **Speed Optimization:** Leverages npm cache hashing and Docker Buildx layer caching to achieve sub-2-minute pipeline runtimes.



Configuration Management & IaC (Ansible)

Automated Server Provisioning, User Security & Reverse Proxy Setup

IAC

📜 Ansible Playbook Architecture

- **Package Management:** Installs Node.js 22 LTS, Docker CE, Nginx reverse proxy, git, and foundational security utilities.
- **User Management:** Creates dedicated `repowise_deployer` service user, configured with restricted passwordless sudo for service restarts.
- **File & Permission Management:** Deploys directory structure `/opt/repowise` (`0750`), secret `.env` (`0600`), Nginx proxy template, and systemd unit.
- **Idempotency:** Safe to re-execute repeatedly without side-effects or configuration drift.

🌐 Nginx & Systemd Service Architecture

- **Reverse Proxy (Port 80 → 5000):** Nginx routes external client traffic to the internal Node.js Express process with WebSocket proxy support.
- **Systemd Daemon (`repowise.service`):** Ensures the backend boots automatically on system restart and self-heals after unexpected crashes.
- **Event Handlers:** Re-loads Nginx and systemd only when templates are modified.

```
Ansible Playbook  --->  Target Host  --->  Install Node 22 + Docker + Nginx
                                    --->  Create User repowise_deployer
                                    --->  Configure Nginx (80 → 5000) & Systemd Unit
```

Kubernetes Orchestration & Zero-Downtime Rollouts

High-Availability Deployment, Rolling Updates & Instant Rollback

KUBERNETES

Deployment Manifest Architecture

- **3 Replicas:** Ensures high availability across cluster worker nodes.
- **Zero-Downtime Strategy:** Configured with `maxSurge: 1` (creates 1 new pod first) and `maxUnavailable: 0` (capacity never drops).
- **Health Probes:** `readinessProbe` shields unready pods from traffic; `livenessProbe` restarts deadlocked pods.
- **Resource Limits:** Requests (128Mi, 100m) and Limits (256Mi, 250m) guarantee cluster stability.

Rolling Update & Instant Rollback Demonstration

- **Rolling Update (v1 → v2):** Executed via `kubectl set image`. Pods update one-by-one with zero user downtime.
- **Instant Rollback:** Reverted immediately via `kubectl rollout undo`, restoring stable v1.0.0 in < 4 seconds if errors occur.
- **Verification:** Validated via `kubectl rollout status` and rollout revision history.

```
Rolling Update: [ReplicaSet v1 (3 pods)] ---> Spawn 1 Pod v2 ---> Readiness OK ---> Terminate 1 Pod v1
Rollback:       kubectl rollout undo ---> Instantly reverts active traffic back to ReplicaSet v1
```

Observability & Key DevOps Lessons Learned

Prometheus + Grafana Telemetry (RED Method) & Engineering Reflection

OBSERVABILITY

⏱️ Service Uptime

99.98%

`avg_over_time(up[1h]) * 100`

Consistent availability surpassing target SLA.

⚡ P95 Latency

42.8 ms

`histogram_quantile(0.95, ...)`

Median p50 response time is 18 ms.

🚨 Error Rate (5xx)

0.02%

`rate(http_requests_total{5xx})`

Well within <1.0% error budget threshold.

🚧 Technical Challenges Overcome

- **Prisma in Alpine Linux:** Resolved missing native libc binaries by adding `libc6-compat` in multi-stage build.
- **Premature Traffic Routing:** Configured `readinessProbe` with 5s delay so DB pools connect before receiving traffic.

🎓 Core Engineering Takeaways

- **Automated Quality Gates:** Vitest and TypeScript checks in CI eliminate release anxiety and bugs.
- **Small Batches Reduce Risk:** Continuous micro-deployments with instant rollbacks make deployments routine.

```
Core Takeaway: DevOps integrates Automated Testing, Declarative IaC, Containerization, and Telemetry into high-velocity delivery.
```